

DEVELOPER PORTFOLIO

안도형

Backend & Infrastructure Engineer —

EMAIL dksehgud1@naver.com

GITHUB github.com/dksehgud

PHONE 010-2613-3584

AWARDS 삼성전자 SW 역량테스트 B형 보유

"개발 단계에서 사용자 불편을 먼저 생각하는 백엔드 개발자"

- 생각하는 방식** 앱이나 웹 서비스를 사용할 때도 이 기능이 뒤에서 어떻게 동작할지 자연스럽게 고민합니다.
- 사용자 상황** 새로고침·이탈·재접속처럼 실제 사용 중 생길 수 있는 상황을 먼저 정리합니다.
- 단순한 구조** 기능을 어렵게 감싸기보다, 문제 크기에 맞는 단순한 구조와 기준을 먼저 선택합니다.
- 협업 문서화** 기획한 사용자 흐름과 예외 상황은 문서로 남겨, 팀원이 같은 기준으로 개발하도록 합니다.

Skills

Java Spring Boot JPA PostgreSQL MySQL Redis Docker
AWS EKS GitLab CI/CD

NAME

안도형 (Ahn Do-hyung)

ROLE FOCUS

Backend / Infra

CORE STACK

Java · Spring Boot · PostgreSQL ·
Redis · AWS EKS

CERTIFICATION

삼성전자 SW 역량테스트 B형

AFFILIATION

가천대학교 졸업
SSAFY 14기 수료 예정

EDUCATION

- 2025.07 ~ 2026.06
삼성 청년 SW 아카데미 (SSAFY)
Java/Python 기반 백엔드 및 클라우드 아키텍처 과정 수료 예정
- 2025.02
가천대학교 컴퓨터공학과 학사 졸업

AWARDS & CERTIFICATIONS

- 2025.08
삼성전자 SW 역량테스트 B형 취득
C++/Java 기반 고성능 자료구조 구현 및 최적화 역량 검증
- 2026.05
우수상 — ReBloom 프로젝트
ReBloom (청소년 우울증 심리 케어 서비스)
- 2026.04
우수상 — 삼성전자 DX부문 기업 협력 프로젝트
S-Catch (AI 적응형 글로벌 웹 크롤링 서비스)
- 2026.02
우수상 — SSAFY 자율 프로젝트
RE:FIT (AI 가상 피팅 중고 공유 플랫폼 - 인프라 전담)

Back-End

Java & Application

Java

Spring Boot 3.x

JPA

Java와 Spring Boot로 인증/인가, 역할 기반 API, 트랜잭션과 상태 관리가 필요한 백엔드 기능을 구현했습니다. JPA는 ReBloom에서 리포트 조회와 데이터 저장 계층을 구성하는 데 사용했습니다.

Data & State

Database & State

PostgreSQL

MySQL

Redis

ReBloom에서는 PostgreSQL, Tget에서는 MySQL을 기준 데이터 저장소로 사용했습니다. Redis는 Tget 대기열의 Waiting, Active, Expired 상태 관리와 DB 기준값 보정에 사용했습니다.

Infra & DevOps

Cloud & Pipeline

Docker

AWS EKS

GitLab CI/CD

Docker로 서비스별 실행 환경을 구성하고 GitLab CI/CD로 반복 빌드와 배포를 자동화했습니다. ReBloom에서는 7개 MSA를 AWS EKS에 배포하고 CloudWatch로 서비스 상태와 DB 연결 상태를 확인했습니다.

PROJECTS LIST

총 4개의 프로젝트 상세 내역

01 ReBloom **우수상**

청소년 우울증 심리 케어를 위한 AIoT 서비스

Spring Boot · EKS · Kafka
· Redis

02 RE:FIT **우수상**

중고 의류 거래 서비스의 배포와 CI/CD를 구축한 프로젝트

Spring Boot · Docker ·
Blue/Green

03 Tget

Redis 대기열로 예매 진입 순서를 제어한 공연 예매 서비스

Spring Boot 3 · Redis ·
MyBatis · k6

04 S-Catch **우수상**

삼성닷컴 상품 정보를 수집·비교한 크롤링 프로젝트

FastAPI · TypeScript ·
Playwright · SSE

인프라 · 백엔드 코어 · 리포트 API

ReBloom

청소년 우울증 심리 케어를 위해 감정 기록과 AIoT 데이터를 연결하는 서비스.

7개 MSA를 EKS에 배포하고 인증/인가, 관계 관리 API, 리포트 API, Redis/ShedLock 스케줄러를 구현했습니다.

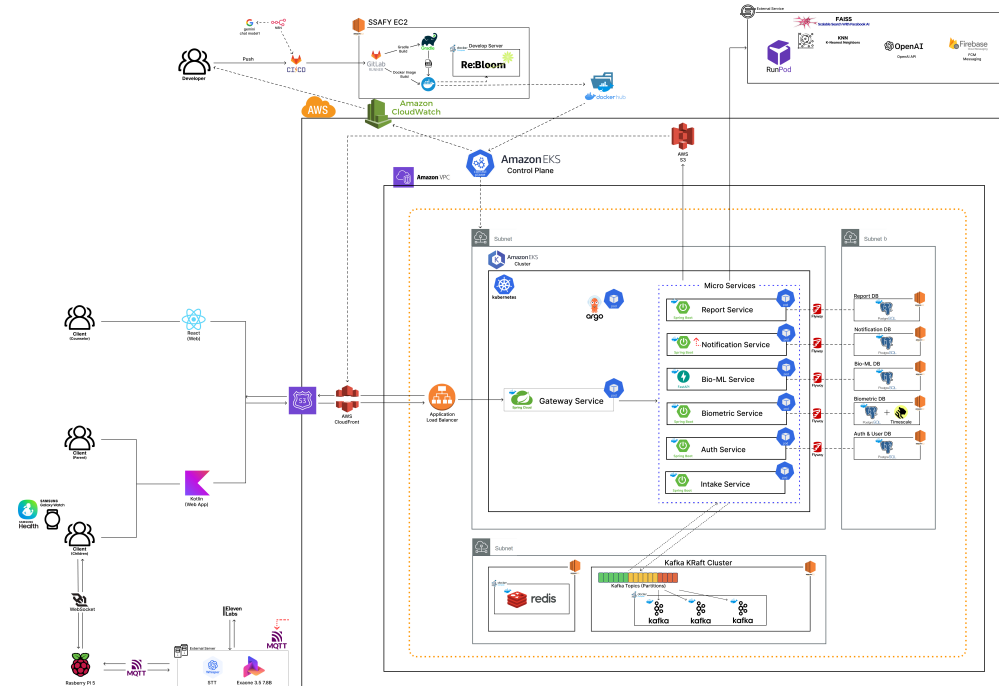
기간 2026.04 ~ 2026.05 (6주)

팀 구성 6명 (BE 2, FE 2, AI 2)

GitHub github.com/dksehgd/ReBloom

시스템 아키텍처

System Architecture



EKS 기반 7개 MSA, Kafka 비동기 이벤트, ShedLock + Redis 분산 스케줄링, ArgoCD GitOps로 서비스 운영 구조를 구성했습니다.

AWS EKS

Kafka

ArgoCD

MySQL

Redis

FastAPI

I ① 상황

ReBloom은 위치 데이터 수집, 생체 분석, 리포트 생성, 보호자 알림이 여러 서비스로 나뉘어 동작하는 구조였습니다. 각 서비스가 따로 움직이면 데이터 중복, 장애 추적, 민감 정보 접근 기준이 쉽게 흔들릴 수 있었습니다.

I ② 과제

- 5분 단위 위치 데이터 수집은 다중 Pod 환경에서도 한 번만 실행되어야 했습니다.
- Kafka로 분리된 분석, 리포트, 알림 과정을 요청 단위로 추적할 수 있어야 했습니다.
- 자녀, 보호자, 상담사 관계에 따라 조회 가능한 정보 범위를 명확히 나눠야 했습니다.

I ③ 행동

- 수집 작업은 한 시점에 한 번만 실행되어야 한다고 판단해 **ShedLock + Redis**로 배치 실행 기준을 잡았습니다.
- 비동기 이벤트도 하나의 요청처럼 추적되어야 한다고 보고 **Kafka** 메시지에 `correlationId`를 넣고 MDC에 연결했습니다.
- API는 기능 기준보다 사용자 관계 기준이 중요하다고 보고 Spring Security와 JWT로 역할별 접근 범위를 분리했습니다.

I ④ 결과 (구현 기준)

중복 방지 다중 Pod 환경에서도 5분 단위 위치 수집 배치가 중복 실행되지 않도록 제어

로그 추적 생체 분석, 리포트 생성, 알림 발송 과정을 correlationId 기준으로 연결

권한 분리 자녀, 보호자, 상담사 관계에 따라 조회 가능한 리포트와 API 범위 분리

운영 기준 CloudWatch 로그와 ArgoCD 상태를 통해 배포 이후 문제 확인 기준 확보

RE:FIT

AI 가상 피팅과 중고 의류 거래 기능을 제공하는 플랫폼.

Backend·Frontend·AI 서버를 Docker로 컨테이너화하고, GitLab CI/CD와 Nginx Blue/Green 기반 배포 자동화를 구현했습니다.

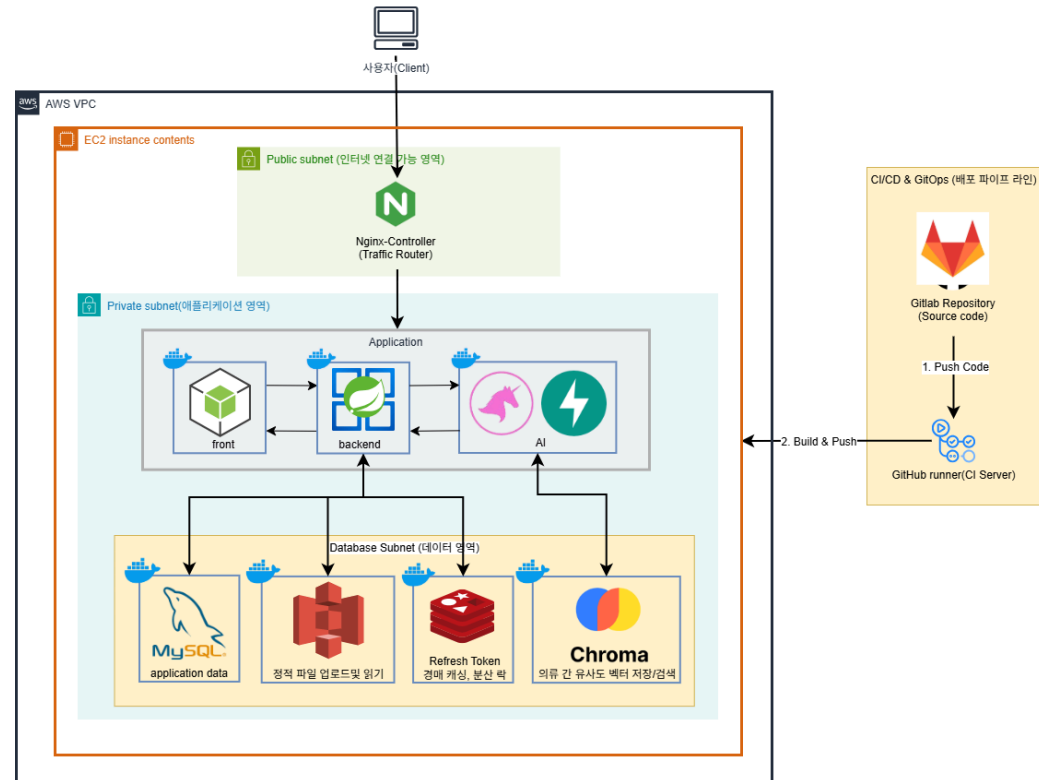
기간 2026.01 ~ 2026.02 (6주)

팀 구성 6명

GitHub github.com/dksehgud/ReFit

시스템 아키텍처

System Architecture



Nginx Blue/Green 배포 자동화, Docker Compose 환경 분리, GitLab CI/CD, Redisson 분산 락, Redis 입찰 검증으로 배포와 동시성 문제를 다뤘습니다.

Docker

Nginx

GitLab CI/CD

Redisson

Redis

MySQL

I ① 문제 정의

배포 때마다 서버가 재시작되면 실시간 경매/채팅 세션이 끊길 수 있었고, AI 서버 호출과 동시 입찰 처리에서 병목과 정합성 문제가 발생할 수 있었습니다. CI 빌드도 반복 실행되며 시간이 길어졌습니다.

I ② 문제 원인

- 단일 환경 배포라 새 버전을 띄우는 동안 기존 서비스가 영향을 받았습니다.
- 동시 입찰과 AI 호출이 물리면 스레드풀과 데이터 정합성 관리가 필요했습니다.
- 매 빌드마다 의존성을 다시 내려받아 CI 시간이 길어졌습니다.

I ③ 해결 과정

- **Nginx Blue/Green 배포 자동화**로 새 버전을 별도 컨테이너에서 실행한 뒤 설정을 갱신했습니다.
- **Redisson 분산 락**과 Redis Sorted Set으로 동시 입찰 검증 흐름을 구현했습니다.
- GitLab **의존성 캐싱**, `rules:changes`, 가상 DB 테스트 환경으로 CI 흐름을 정리했습니다.

I ④ 결과 (GitLab API 실측 기준)

532회 GitLab 파이프라인 총 실행

205회 Blue/Green 배포 작업 실행

87.0% 2월 파이프라인 성공률 (안정화)

83.1초 성공 배포 소요 중앙값

대기열 제어 · Redis · 상태 정합성 · 부하 테스트

Tget

공연 예매 오픈 시 한 번에 물리는 요청을 Redis 대기열로 완충한 서비스.

Waiting, Active, Expired 상태를 기준으로 입장 순서를 제어하고, 이탈 사용자 토큰 만료와 Redis-DB active count 보정까지 구현했습니다.

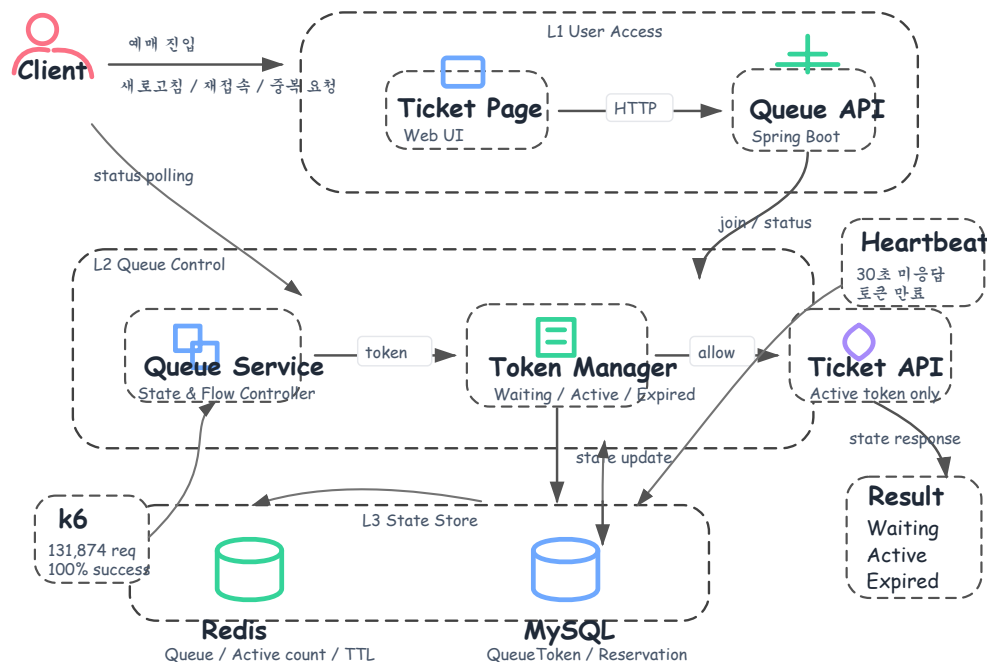
기간 2025.11 ~ 2025.12 (4주)

팀 구성 2명 (BE 1, FE 1)

GitHub github.com/dksehgud/Tget

대기열 구조

Queue Architecture



대기열은 빠른 상태 제어가 필요한 Redis와 기준 데이터가 필요한 MySQL을 분리하고, Heartbeat와 Scheduler로 이탈·재시작 상황의 상태 불일치를 보정했습니다.

Spring Boot

Redis

MySQL

MyBatis

Scheduler

k6

I ① 문제 정의

예매 오픈 시 요청이 한 번에 DB로 들어오면 커넥션이 부족해지고, 사용자가 이탈해도 active 토큰이 남아 다음 사용자의 진입이 늦어질 수 있었습니다.

I ② 문제 원인

- 예매 요청이 완충 없이 DB로 직접 유입되어 커넥션 부담이 커졌습니다.
- 서버 재시작 시 Redis active count는 초기화되지만 DB 상태는 유지되어 정합성이 어긋날 수 있었습니다.
- 이탈 사용자를 감지하지 못하면 active 토큰이 계속 남을 수 있었습니다.

I ③ 해결 과정

- Redis + MySQL 대기열 토큰 구조**로 Waiting/Active/Expired 상태를 관리했습니다.
- Heartbeat 기반 만료**로 30초 동안 응답이 없는 active 토큰을 만료 처리했습니다.
- QueueScheduler**가 30초 주기로 DB active count를 Redis에 동기화하도록 구현했습니다.
- 부하 테스트로 대기열 동시성 제어, 요청 성공률, 순서 유지 흐름을 확인했습니다.

I ④ 결과 (테스트 환경 기준)

131,874회 부하 테스트 총
요청 수

100% 테스트 요청 성공률

197 RPS 테스트 최대 처리량

95.2% 대기열 순서 유지율 개
선

보조 프로젝트 · 크롤링 자동화 · SSE

S-Catch

삼성닷컴 글로벌 상품 정보를 수집하고 기준 데이터와 비교한 크롤링 프로젝트.
FastAPI 작업 API, Playwright 수집, SSE 진행 상태 조회, Import/Diff 기능을 구현했습니다.

기간 2026.02 ~ 2026.04 (6주)

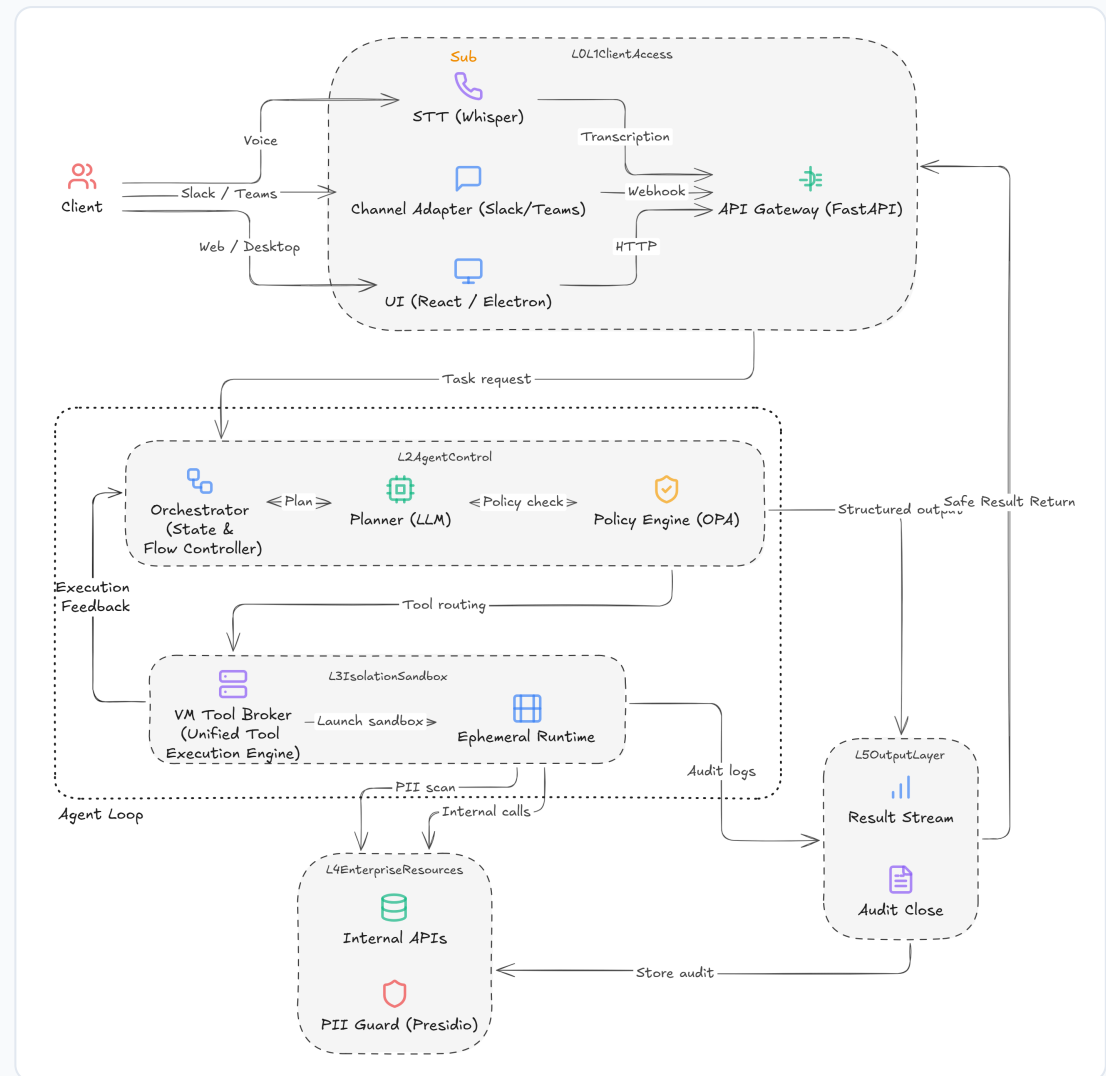
팀 구성 6명

GitHub github.com/dksehgd/s-catch

안도형 포트폴리오

구현 범위

Additional Project



주요 어필 프로젝트는 아니지만, 국가별 페이지 구조 차이를 고려해 수집 작업 생성, 상태 조회, 결과 비교 기능을 구현한 경험으로 정리했습니다.

FastAPI

Playwright

TypeScript

SSE

Import/Diff

PROJECT S-CATCH · ADDITIONAL

THANK YOU

로그와 지표로 문제를 추적하고, 다시 발생하지 않는 구조로 개선하는 개발자 안도형이었습니다.

